

Divide and Conquer

Binary Search & Merge Strategy

Definitions • Examples • Algorithms • Flowcharts • Python • Software Applications

DIVIDE

Break the problem into smaller parts.

CONQUER

Solve smaller subproblems.

COMBINE

Build the final solution.

Learning Outcomes

Students will be able to:

Explain

Define Divide and Conquer and identify Divide, Conquer, Combine and the Base Case.

Implement

Write algorithms and Python programs for Binary Search and Merge Strategy.

Analyze

Interpret flowcharts and analyze time and space complexity.

Divide and Conquer: Definition

A fundamental algorithm design paradigm

Divide and Conquer is an algorithmic strategy in which a problem is divided into smaller subproblems, each subproblem is solved, and the partial solutions are combined to solve the original problem.

1. Divide

Split the original problem into smaller manageable subproblems.

2. Conquer

Solve each subproblem, often recursively, until a base case is reached.

3. Combine

Combine subproblem results to produce the final answer.

General Algorithmic Pattern

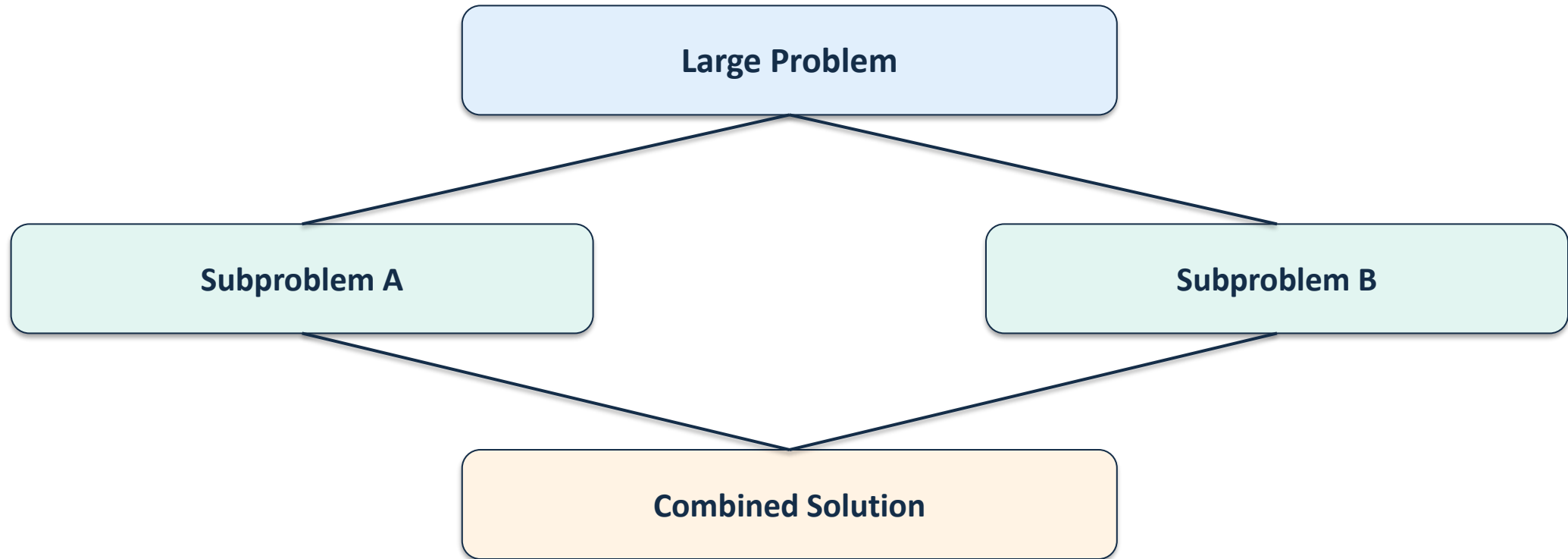
The structure behind Divide and Conquer algorithms

```
Solve(problem):  
    if problem is small enough:  
        solve directly  
        return result  
  
    divide problem into subproblems  
  
    solve subproblems recursively  
  
    combine the results  
  
    return final result
```

- Base Case stops further division.
- Each recursive call handles a smaller problem.
- The subproblems follow the same general structure.
- The combine stage depends on the particular algorithm.

Example of the Concept

From one large problem to smaller problems



Real-world idea: A large dataset can be divided into smaller groups, processed separately, and the partial results can later be combined.

Binary Search: Definition

A Divide and Conquer searching technique

Binary Search repeatedly examines the middle of a sorted search space and discards the half that cannot contain the target.

10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----

The middle element is compared with the target. Only one half remains after an unsuccessful comparison.

Requirement

The data must be sorted.

Benefit

Approximately half of the remaining data can be eliminated each step.

Binary Search Example

Find target = 70

Sorted array: [10, 20, 30, 40, 50, 60, 70, 80, 90]

10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----

Step 1: Middle = 50. Since $70 > 50$, discard the left half.

60	70	80	90
----	----	----	----

Step 2: Middle = 70. The target is found.

Key Observation

The search range becomes much smaller instead of checking every element sequentially.

Algorithm: Binary Search

Iterative version

```
BINARY_SEARCH(array,
target)

low ← 0
high ← length(array) - 1

WHILE low ≤ high
    mid ← (low + high) // 2

    IF array[mid] = target
        RETURN mid
    ELSE IF array[mid] <
target
        low ← mid + 1
    ELSE
        high ← mid - 1

RETURN -1
```

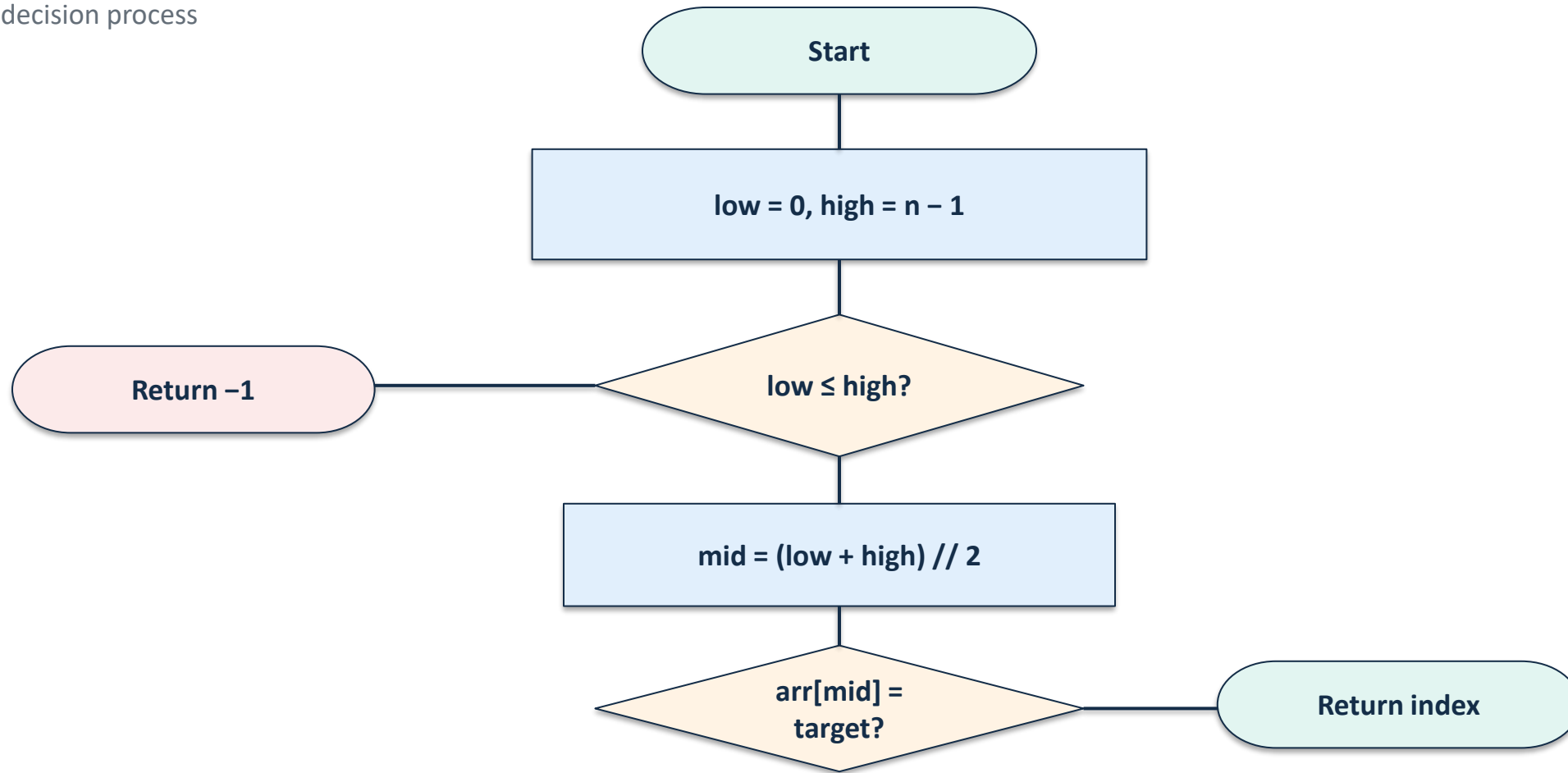
Binary Search Algorithm

```
1. Def. binary Search (A, x):
2. n = len (A)
3. beg = 0
4. end = n - 1
5. result = -1
6. While (beg <= end):
7.     mid = (beg + end) / 2
8.     If (A[mid] <= x):
9.         beg = mid + 1
10.        result = mid
11.    Else:
12.        end = mid - 1
13. Return result
```

- low marks the start of the current range.
- high marks the end of the current range.
- mid is the middle position.
- One half is discarded after each comparison.
- Return -1 when the target does not exist.

Flowchart: Binary Search

Visual decision process



If the middle is not the target, compare values, update low or high, and repeat.

Python Program: Binary Search

Searching a sorted list

```
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

numbers = [10, 20, 30, 40, 50, 60, 70]
print(binary_search(numbers, 60))
```

- The input list is sorted.
- // performs integer division.
- The target is compared with the middle value.
- The active search range is reduced until found or empty.

Binary Search: Complexity Analysis

Why repeated halving is efficient

Best Case: $O(1)$

The target is found immediately at the first middle position.

Average/Worst: $O(\log n)$

Each unsuccessful comparison approximately halves the remaining search space.

Iterative Space: $O(1)$

Only a fixed number of variables such as low, high, and mid are required.

Binary Search: Software Applications

Where range reduction is useful

Database Indexing

Ordered indexes and tree-based search structures use related efficient search principles.

Libraries & APIs

Efficient lookup within sorted collections and ordered data.

Debugging

Binary-search ideas can help identify the point where a regression was introduced.

Numerical Computing

Finding thresholds or values within a monotonic range.

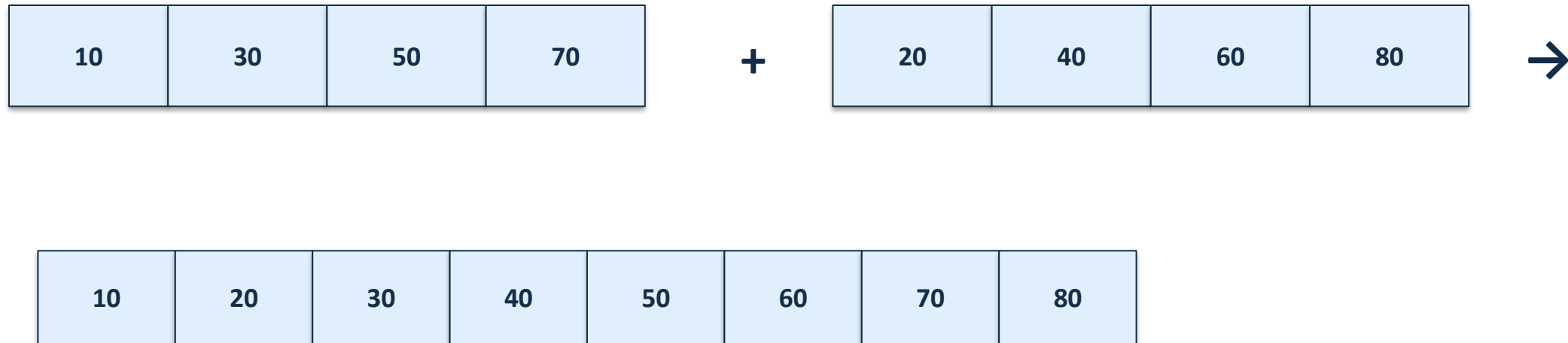
Applications & Games

Searching ordered score tables, configuration ranges, and structured datasets.

Merge Strategy: Definition

Combining two sorted sequences

The Merge Strategy combines two sorted sequences into one sorted sequence by repeatedly selecting the smallest current element.



Merge is the key combine operation used by Merge Sort and large-scale external sorting.

Merge Strategy: Step-by-Step Example

Merge [10, 30, 50] and [20, 40, 60]

10	30	50
----	----	----

20	40	60
----	----	----

- Compare 10 and 20 → select 10.
- Compare 30 and 20 → select 20.
- Compare 30 and 40 → select 30.
- Continue until one sequence is exhausted.
- Copy the remaining elements of the other sequence.

10	20	30	40	50	60
----	----	----	----	----	----

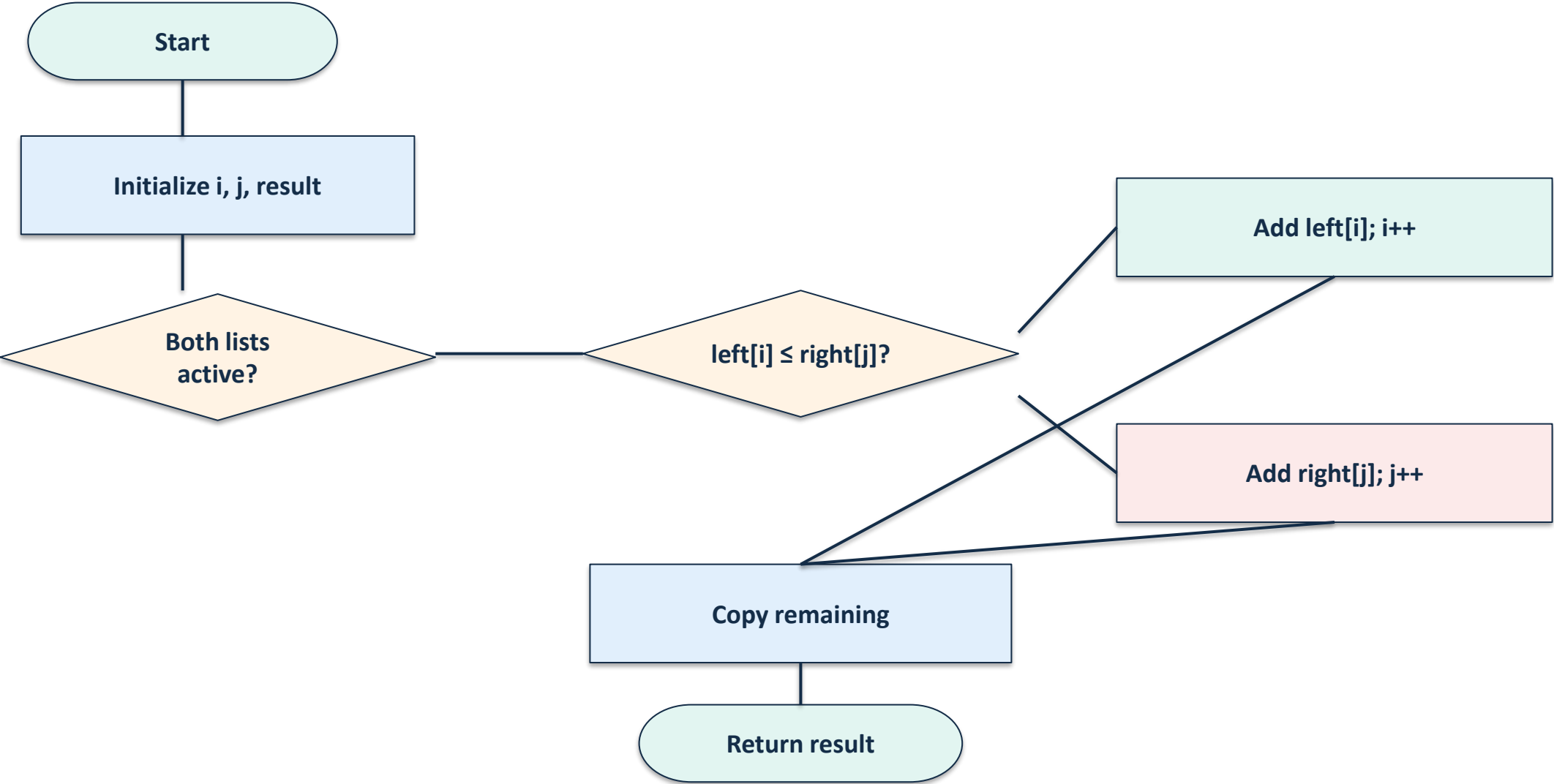
Algorithm: Merge Two Sorted Lists

```
1. Start
2. Create an empty list 'result'
3. Set i to 0
4. Set j to 0
5. While i<(len(left_arr)) AND j<(len(right_arr)),
   repeat:
     1. If left[i] ≤ right[j]:
       1. Add left [i] to result
       2. Set i to i+1
     2. Else
       1. Add right [j] to result
       2. Set j to j+1
6. While i<len(left):
     1. Add left[i] to result
     2. Set i to i+1
7. While j<len(right):
     1. Add right[j] to result
     2. Set j to j+1
8. Return Result
9. Stop
```

- i tracks the left sequence.
- j tracks the right sequence.
- The smaller current value is selected.
- Each input element is processed once.

Flowchart: Merge Strategy

How two sorted lists are combined



Merge Strategy: Complexity and Applications

Processing ordered data efficiently

Time: $O(n + m)$

Each element of the two lists is processed at most once.

Space: $O(n + m)$

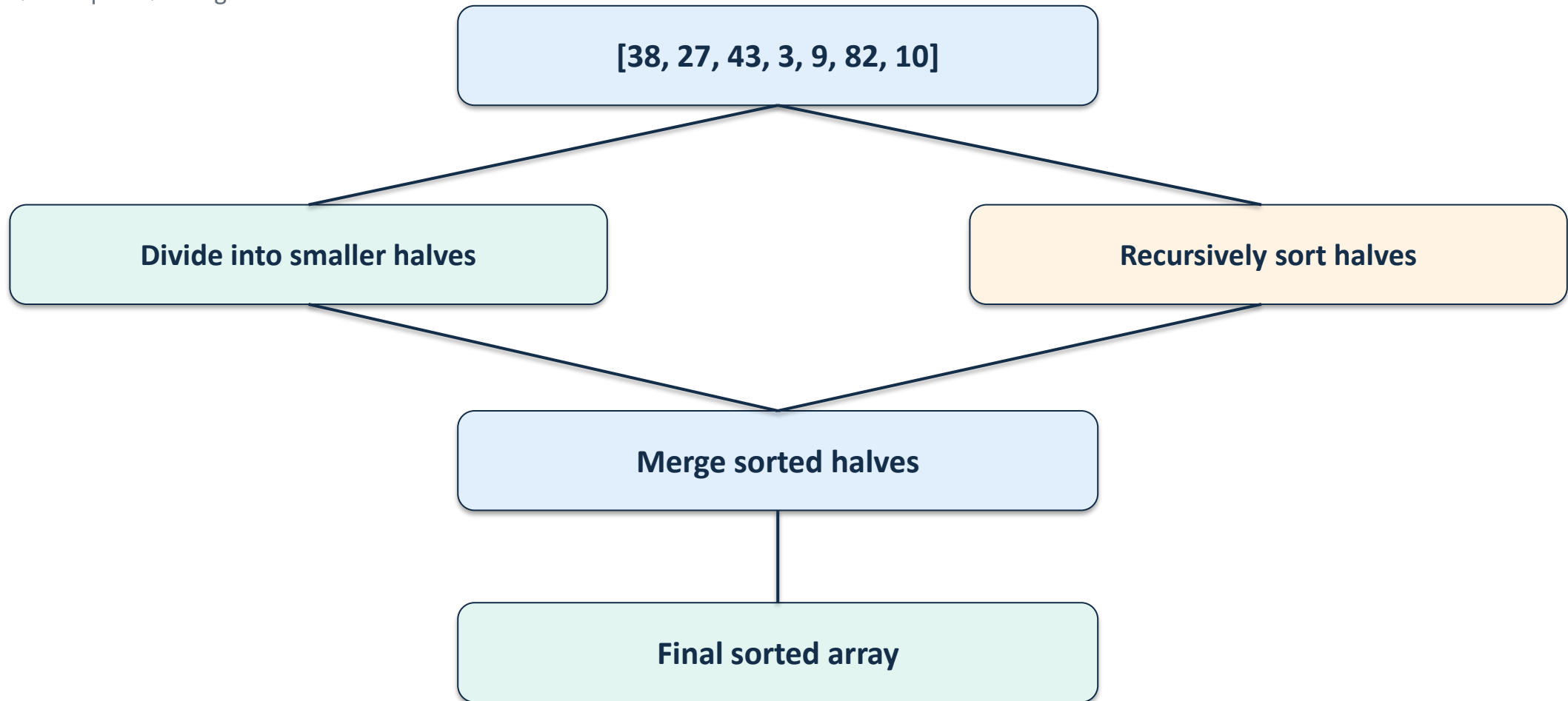
A separate result list commonly stores the merged output.

Software Uses

Merge Sort, database query processing, external sorting, search/data pipelines, and distributed data processing.

Merge Sort: Divide and Conquer in Action

Divide → Conquer → Merge



This algorithm clearly demonstrates all three stages of Divide and Conquer.

Summary and Discussion Questions

Key relationships among the topics

Divide and Conquer

General paradigm: divide a problem, solve smaller problems, and combine results.

Binary Search

Uses repeated division of a sorted search space. Typical complexity: $O(\log n)$.

Merge Strategy

Combines sorted sequences. Typical merge time: $O(n + m)$.

- Why must Binary Search use sorted data?
- What is the difference between Divide and Conquer and Merge Strategy?
- Why does Binary Search reduce complexity compared with Linear Search?
- How does Merge Sort use both Divide and Conquer and the Merge Strategy?
- Name two software systems where these concepts are useful.